

```
library(tidyverse)
library(glmnet)
library(rsample)
library(randomForest)
library(xgboost)
library(yardstick)
library(ggplot2)
library(shapviz)
```

1. LOAD DATA & VARIABLE GROUPS

```
data <- read.csv("hpi_clean_full.csv")
```

```
set.seed(42)
```

```
# 80/20 train-test split (stratified by income_quartile)
split <- initial_split(data, prop = 0.80, strata = income_quartile)
train_data <- training(split)
test_data <- testing(split)

cat("Training rows:", nrow(train_data), "\n")
```

```
## Training rows: 5610
```

```
cat("Test rows:      ", nrow(test_data), "\n")
```

```
## Test rows:      1404
```

```
var_groups <- readRDS("variable_groups.rds")
indicators_raw <- var_groups$indicators_raw
domain_scores <- var_groups$domain_scores
demo_vars <- var_groups$demo_vars
response_var <- var_groups$response_var

# Re-apply factor levels lost during CSV write
train_data <- train_data %>%
  mutate(
    income_quartile = factor(income_quartile,
                             levels = c("Q1_low", "Q2", "Q3", "Q4_high")),
    hpi_quartile = factor(hpi_quartile),
    urban_type = factor(urban_type)
  )
```

```
test_data <- test_data %>%
  mutate(
    income_quartile = factor(income_quartile,
                             levels = c("Q1_low", "Q2", "Q3", "Q4_high")),
    hpi_quartile    = factor(hpi_quartile),
    urban_type      = factor(urban_type)
  )
```

2. CREATE REGION VARIABLE FROM COUNTIES

```
bay_area <- c(
  "Alameda", "Contra Costa", "Marin", "Napa", "San Francisco",
  "San Mateo", "Santa Clara", "Solano", "Sonoma"
)
socal <- c(
  "Los Angeles", "Orange", "Riverside", "San Bernardino",
  "San Diego", "Imperial", "Ventura"
)
central_valley <- c(
  "Fresno", "Kern", "Kings", "Madera", "Merced",
  "San Joaquin", "Stanislaus", "Tulare"
)
norcal <- c(
  "Butte", "Colusa", "Del Norte", "Glenn", "Humboldt", "Lake",
  "Lassen", "Mendocino", "Modoc", "Plumas", "Shasta", "Sierra",
  "Siskiyou", "Tehama", "Trinity"
)

assign_region <- function(county_name) {
  case_when(
    county_name %in% bay_area ~ "Bay Area",
    county_name %in% socal ~ "SoCal",
    county_name %in% central_valley ~ "Central Valley",
    county_name %in% norcal ~ "NorCal",
    TRUE ~ "Other CA"
  )
}

train_data <- train_data %>%
  mutate(region = factor(assign_region(county)))

test_data <- test_data %>%
  mutate(region = factor(assign_region(county)))

final_df <- data %>%
```

```
mutate(region = factor(assign_region(county)))

cat("Region distribution in training data:\n")
```

Region distribution in training data:

```
print(table(train_data$region))
```

```
##
##      Bay Area Central Valley      NorCal      Other CA      SoCal
##      1170          497          158          595          3190
```

3. LASSO — VARIABLE SELECTION

```
lasso_vars <- c(indicators_raw, demo_vars)

x_train <- train_data %>%
  select(all_of(lasso_vars)) %>%
  drop_na() %>%
  as.matrix()

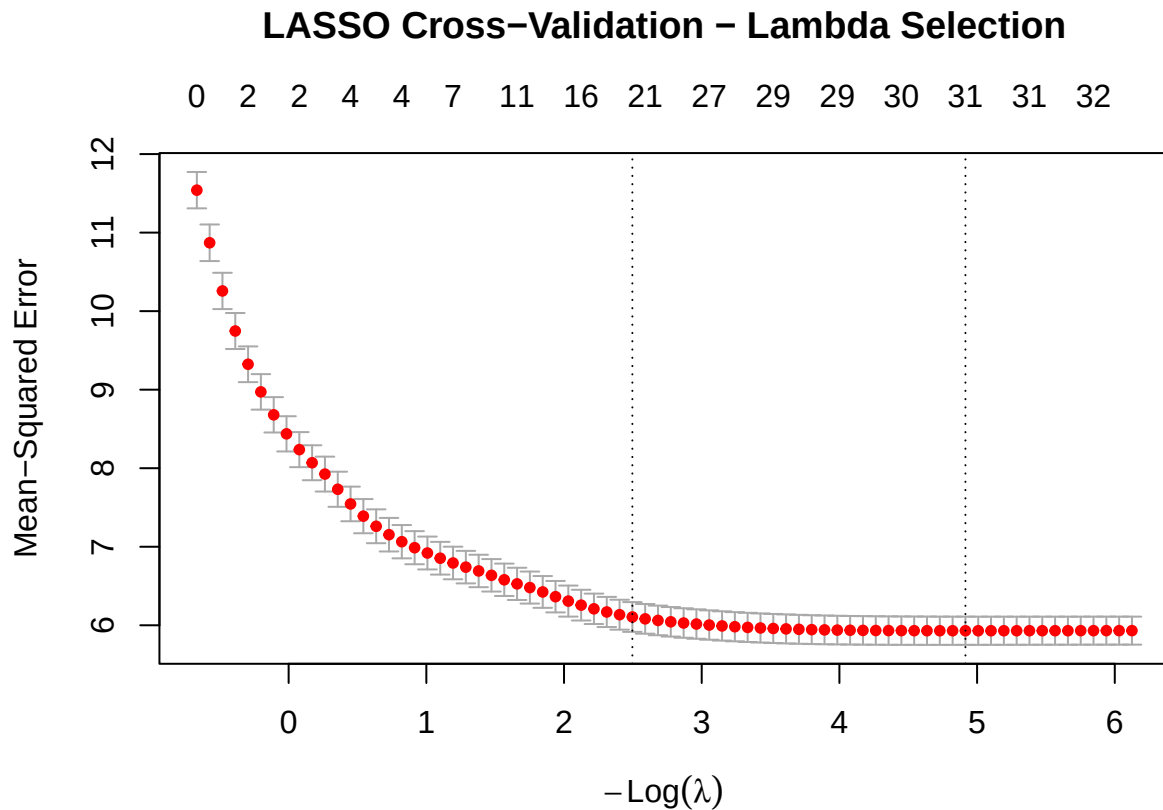
y_train <- train_data %>%
  filter(complete.cases(select(., all_of(lasso_vars)))) %>%
  pull(leb_cdc)

x_test <- test_data %>%
  select(all_of(lasso_vars)) %>%
  drop_na() %>%
  as.matrix()

y_test <- test_data %>%
  filter(complete.cases(select(., all_of(lasso_vars)))) %>%
  pull(leb_cdc)

# Cross-validated LASSO to find optimal lambda
set.seed(42)
lasso_cv <- cv.glmnet(
  x      = x_train,
  y      = y_train,
  alpha  = 1,          # alpha=1 is LASSO (alpha=0 is Ridge)
  nfolds = 10,
  standardize = TRUE    # standardize predictors (important for LASSO)
)
```

```
plot(lasso_cv)
title("LASSO Cross-Validation - Lambda Selection", line = 3)
```



```
# Best lambda (1 SE rule - more parsimonious model)
best_lambda <- lasso_cv$lambda.1se
cat("\nBest lambda (1SE):", round(best_lambda, 4), "\n")
```

```
##
## Best lambda (1SE): 0.0824
```

```
# Extract non-zero coefficients = selected variables
lasso_coefs <- coef(lasso_cv, s = "lambda.1se")
selected_vars <- rownames(lasso_coefs)[which(lasso_coefs != 0)]
selected_vars <- selected_vars[selected_vars != "(Intercept)"]

cat("\nLASSO selected", length(selected_vars), "variables:\n")
```

```
##
## LASSO selected 21 variables:
```

```
print(selected_vars)
```

```
## [1] "abovepoverty" "automobile" "bachelorsed" "dieselpm"
```

```
## [5] "employed"      "h20contam"      "homeownership"  "houstorepair"
## [9] "inpreschool"   "ownsevere"      "percapitaincome" "uncrowded"
## [13] "voting"        "pop"            "pctgqtract"     "white_pct"
## [17] "black_pct"     "asian_pct"      "multiple_pct"   "native_am_pct"
## [21] "pacific_isl_pct"
```

```
# LASSO predictions on test set
lasso_preds <- predict(lasso_cv, newx = x_test, s = "lambda.1se") %>% as.vector()

lasso_rmse <- sqrt(mean((y_test - lasso_preds)^2))
lasso_r2    <- cor(y_test, lasso_preds)^2
cat("\nLASSO Test RMSE:", round(lasso_rmse, 3))
```

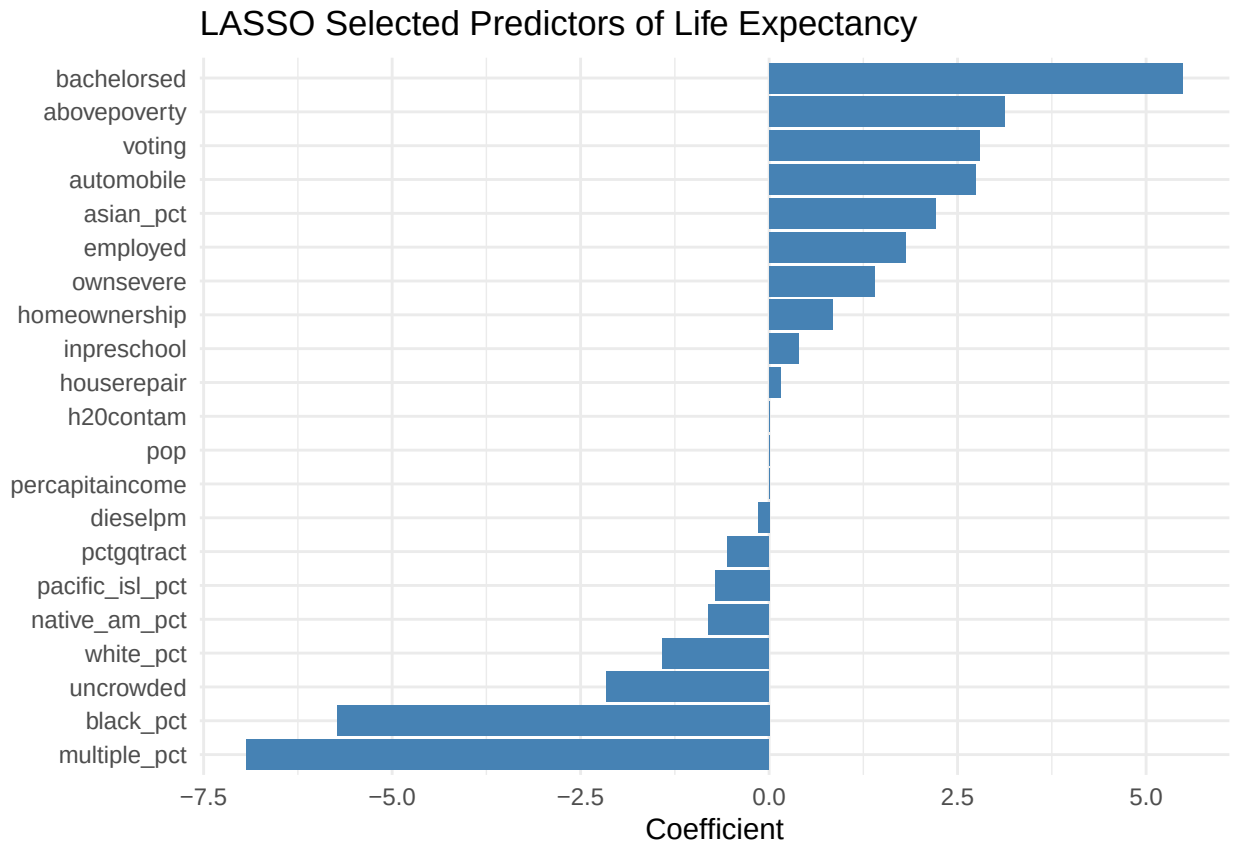
```
##
## LASSO Test RMSE: 2.447
```

```
cat("\nLASSO Test R2: ", round(lasso_r2, 3), "\n")
```

```
##
## LASSO Test R2: 0.464
```

```
# Coefficient plot - shows direction and magnitude of selected predictors
lasso_coef_df <- data.frame(
  variable = selected_vars,
  coefficient = as.vector(lasso_coefs[selected_vars, ])
) %>% arrange(coefficient)

ggplot(lasso_coef_df, aes(x = reorder(variable, coefficient), y = coefficient)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "LASSO Selected Predictors of Life Expectancy",
       x = NULL, y = "Coefficient") +
  theme_minimal()
```



4. 2-WAY ANOVA — REGION x INCOME LEVEL

```
anova_df <- final_df %>%
  select(leb_cdc, region, income_quartile) %>%
  drop_na()

# Check cell counts (need reasonable n per cell)
cat("\nCell counts (region x income_quartile):\n")

##
## Cell counts (region x income_quartile):

print(table(anova_df$region, anova_df$income_quartile))

##
##           Q1_low   Q2   Q3  Q4_high
```

```
## Bay Area      81  216  380    767
## Central Valley 304  188  107     21
## NorCal        46   98   49      4
## Other CA     124  202  286    138
## SoCal        1199 1050  931    823
```

```
# Fit 2-way ANOVA with interaction
anova_model <- aov(leb_cdc ~ region * income_quartile, data = anova_df)
cat("\n2-Way ANOVA Results:\n")
```

```
##
## 2-Way ANOVA Results:
```

```
print(summary(anova_model))
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## region          4   7491    1873 240.907 < 2e-16 ***
## income_quartile  3  18264    6088 783.117 < 2e-16 ***
## region:income_quartile 12   322      27   3.457 4.22e-05 ***
## Residuals      6994  54370         8
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Post-hoc: Tukey HSD to see which pairs differ significantly
tukey_results <- TukeyHSD(anova_model)
cat("\nTukey HSD - Region:\n")
```

```
##
## Tukey HSD - Region:
```

```
print(tukey_results$region)
```

```
##              diff          lwr          upr          p adj
## Central Valley-Bay Area -3.6831838 -4.0484551 -3.3179126 5.465184e-12
## NorCal-Bay Area        -3.8443480 -4.4221482 -3.2665479 5.465184e-12
## Other CA-Bay Area      -1.7003494 -2.0427576 -1.3579412 5.465184e-12
## SoCal-Bay Area         -1.1031154 -1.3366452 -0.8695855 5.465184e-12
## NorCal-Central Valley  -0.1611642 -0.7833530  0.4610245 9.550271e-01
## Other CA-Central Valley 1.9828344  1.5699068  2.3957620 5.465184e-12
## SoCal-Central Valley   2.5800684  2.2517363  2.9084006 5.465184e-12
## Other CA-NorCal        2.1439986  1.5349509  2.7530464 5.509593e-12
## SoCal-NorCal           2.7412327  2.1860462  3.2964191 5.465184e-12
## SoCal-Other CA         0.5972340  0.2945423  0.8999257 7.510181e-07
```

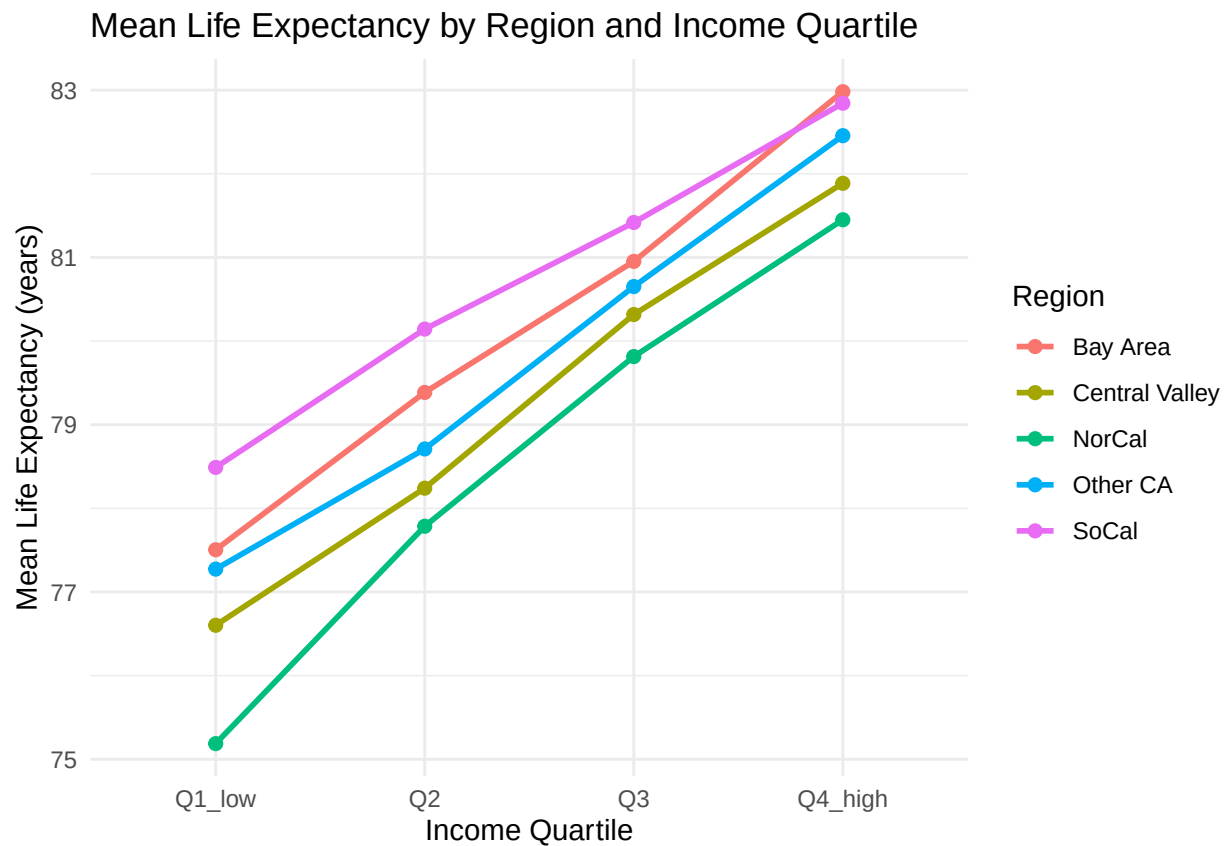
```
cat("\nTukey HSD - Income Quartile:\n")
```

```
##
## Tukey HSD - Income Quartile:
```

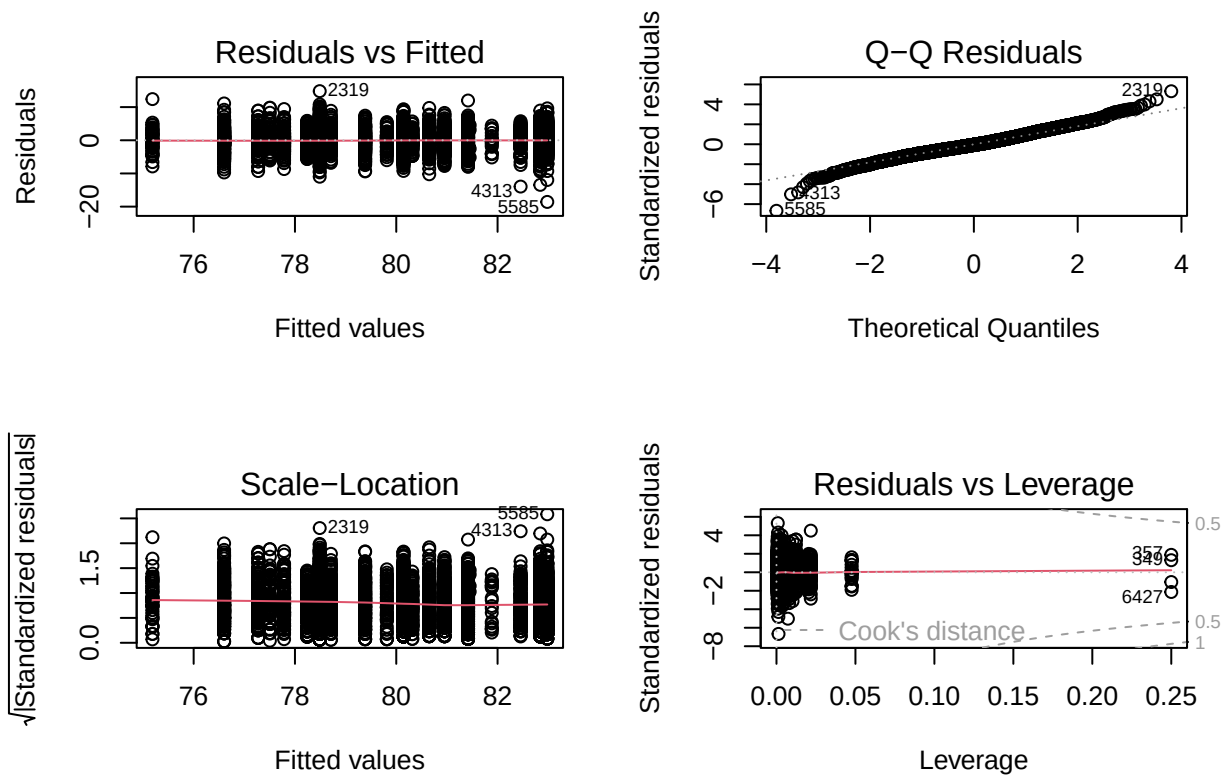
```
print(tukey_results$income_quartile)
```

```
##               diff      lwr      upr      p adj
## Q2-Q1_low      1.457480 1.215548 1.699411 5.465184e-12
## Q3-Q1_low      2.717527 2.475561 2.959493 5.465184e-12
## Q4_high-Q1_low 4.005469 3.763503 4.247435 5.465184e-12
## Q3-Q2          1.260047 1.018081 1.502013 5.465184e-12
## Q4_high-Q2     2.547989 2.306023 2.789955 5.465184e-12
## Q4_high-Q3     1.287942 1.045942 1.529942 5.465184e-12
```

```
# Interaction plot - visualize how income effect varies by region
anova_df %>%
  group_by(region, income_quartile) %>%
  summarise(mean_leb = mean(leb_cdc, na.rm = TRUE), .groups = "drop") %>%
  ggplot(aes(x = income_quartile, y = mean_leb,
             color = region, group = region)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(title = "Mean Life Expectancy by Region and Income Quartile",
       x = "Income Quartile", y = "Mean Life Expectancy (years)",
       color = "Region") +
  theme_minimal()
```




```
# Check ANOVA assumptions
par(mfrow = c(2, 2))
plot(anova_model)
```



```
par(mfrow = c(1, 1))
```

5. K-MEANS CLUSTERING

```
# Use LASSO-selected variables for clustering
cluster_vars <- intersect(selected_vars, colnames(train_data))

cluster_df <- final_df %>%
  select(geo_id, all_of(cluster_vars)) %>%
  drop_na()

# Scale variables (k-means is distance-based, scaling is required)
```

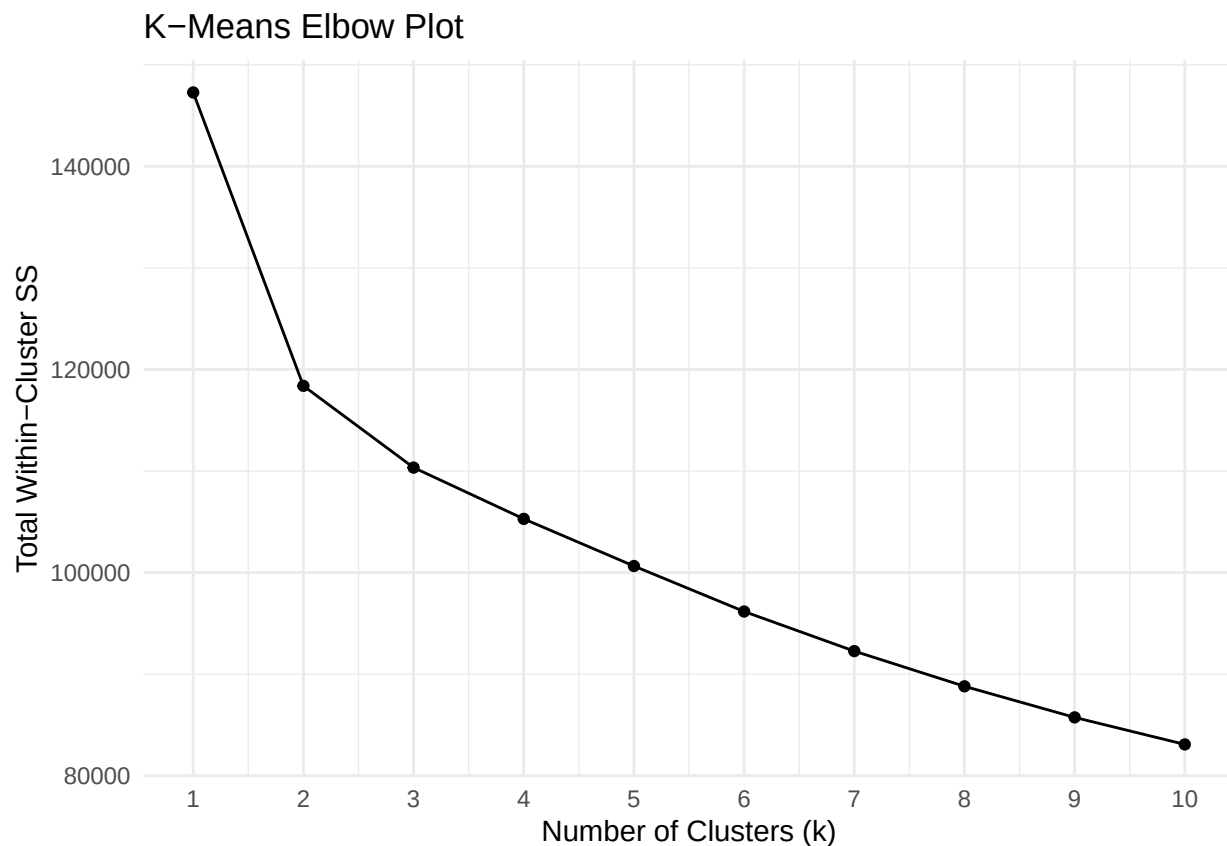
```

cluster_scaled <- cluster_df %>%
  select(-geo_id) %>%
  scale()

# Elbow plot - find optimal number of clusters
set.seed(42)
wss <- map_dbl(1:10, ~ {
  kmeans(cluster_scaled, centers = .x, nstart = 25, iter.max = 100)$tot.withinss
})

tibble(k = 1:10, wss = wss) %>%
  ggplot(aes(x = k, y = wss)) +
  geom_line() + geom_point() +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "K-Means Elbow Plot",
       x = "Number of Clusters (k)", y = "Total Within-Cluster SS") +
  theme_minimal()

```



```

# Fit k-means with chosen k (adjust after reviewing elbow plot)
k_chosen <- 4

set.seed(42)
kmeans_fit <- kmeans(cluster_scaled, centers = k_chosen,
                     nstart = 25, iter.max = 100)

```

```

# Add cluster labels back to data
cluster_df <- cluster_df %>%
  mutate(cluster = factor(kmeans_fit$cluster))

final_df <- final_df %>%
  left_join(cluster_df %>% select(geo_id, cluster), by = "geo_id")

cat("\nCluster sizes:\n")

```

```

##
## Cluster sizes:

```

```

print(table(cluster_df$cluster))

```

```

##
##      1      2      3      4
## 2256 1958 1066 1734

```

```

# Profile clusters by mean life expectancy
final_df %>%
  group_by(cluster) %>%
  summarise(
    n          = n(),
    mean_leb   = mean(leb_cdc, na.rm = TRUE),
    mean_hpi   = mean(hpi, na.rm = TRUE),
    mean_income = mean(percapitaincome, na.rm = TRUE)
  ) %>%
  arrange(mean_leb) %>%
  print()

```

```

## # A tibble: 4 x 5
##   cluster      n mean_leb mean_hpi mean_income
##   <fct>   <int>   <dbl>   <dbl>     <dbl>
## 1 4         1734    78.2   -0.663    18780.
## 2 1         2256    79.5   -0.109    30282.
## 3 3         1066    81.8    0.293    43041.
## 4 2         1958    82.5    0.638    63459.

```

6. RANDOM FOREST

```

# Use LASSO-selected variables as predictors
rf_vars <- selected_vars

```

```

rf_train <- train_data %>%
  select(all_of(c(rf_vars, "leb_cdc"))) %>%
  drop_na()

rf_test <- test_data %>%
  select(all_of(c(rf_vars, "leb_cdc"))) %>%
  drop_na()

set.seed(42)
rf_model <- randomForest(
  x      = rf_train %>% select(-leb_cdc),
  y      = rf_train$leb_cdc,
  ntree  = 500,
  mtry   = floor(sqrt(length(rf_vars))), # default for regression
  importance = TRUE
)

print(rf_model)

```

```

##
## Call:
## randomForest(x = rf_train %>% select(-leb_cdc), y = rf_train$leb_cdc,      ntree = 500, mtry = floor
##               Type of random forest: regression
##               Number of trees: 500
## No. of variables tried at each split: 4
##
##               Mean of squared residuals: 5.642901
##               % Var explained: 51.16

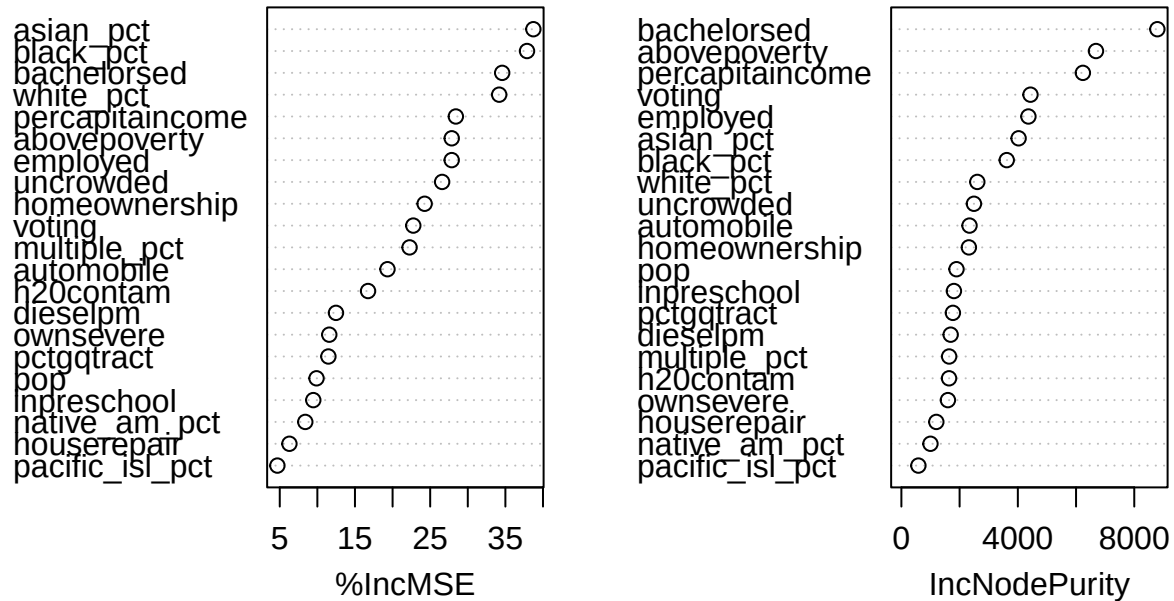
```

```

# Variable importance plot
varImpPlot(rf_model, main = "Random Forest - Variable Importance")

```

Random Forest – Variable Importance



```
# Test set predictions
rf_preds <- predict(rf_model, newdata = rf_test %>% select(-leb_cdc))

rf_rmse <- sqrt(mean((rf_test$leb_cdc - rf_preds)^2))
rf_r2 <- cor(rf_test$leb_cdc, rf_preds)^2
cat("\nRandom Forest Test RMSE:", round(rf_rmse, 3))
```

```
##
## Random Forest Test RMSE: 2.367
```

```
cat("\nRandom Forest Test R^2: ", round(rf_r2, 3), "\n")
```

```
##
## Random Forest Test R^2: 0.5
```

7. XGBOOST

```

# XGBoost requires numeric matrices - factors must be dummy-encoded
xgb_vars <- selected_vars # already numeric from LASSO step

xgb_train_x <- train_data %>%
  select(all_of(xgb_vars)) %>%
  drop_na() %>%
  as.matrix()

xgb_train_y <- train_data %>%
  filter(complete.cases(select(., all_of(xgb_vars)))) %>%
  pull(leb_cdc)

xgb_test_x <- test_data %>%
  select(all_of(xgb_vars)) %>%
  drop_na() %>%
  as.matrix()

xgb_test_y <- test_data %>%
  filter(complete.cases(select(., all_of(xgb_vars)))) %>%
  pull(leb_cdc)

dtrain <- xgb.DMatrix(data = xgb_train_x, label = xgb_train_y)
dtest  <- xgb.DMatrix(data = xgb_test_x, label = xgb_test_y)

params <- list(
  objective = "reg:squarederror",
  learning_rate = 0.05,
  max_depth = 4,
  subsample = 0.8
)

# Cross-validated XGBoost
set.seed(42)
xgb_cv <- xgb.cv(
  data = dtrain,
  nrounds = 500,
  nfold = 10,
  params = params,
  early_stopping_rounds = 20,
  verbose = 0
)

best_nrounds <- which.min(xgb_cv$evaluation_log$test_rmse_mean)
cat("\nXGBoost best nrounds:", best_nrounds, "\n")

```

```

##
## XGBoost best nrounds: 222

```

```

# Fit final XGBoost model
set.seed(42)
xgb_model <- xgb.train(
  data = dtrain,
  nrounds = best_nrounds,

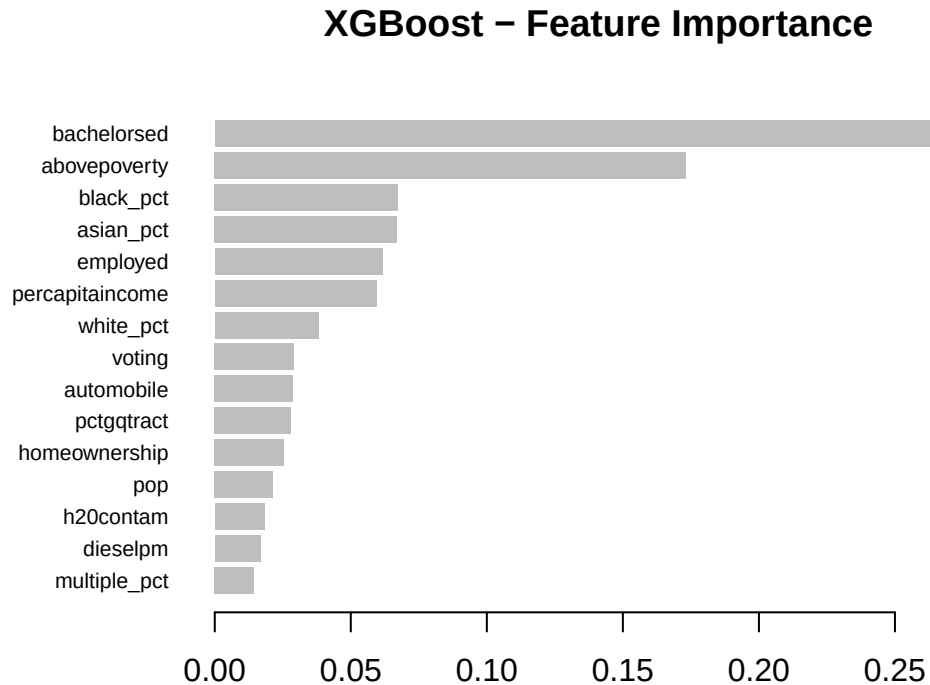
```

```

params = params,
verbose = 0
)

# Feature importance
xgb_importance <- xgb.importance(model = xgb_model)
xgb.plot.importance(xgb_importance, top_n = 15,
                    main = "XGBoost - Feature Importance")

```



```

# Test set predictions
xgb_preds <- predict(xgb_model, dtest)

xgb_rmse <- sqrt(mean((xgb_test_y - xgb_preds)^2))
xgb_r2 <- cor(xgb_test_y, xgb_preds)^2
cat("\nXGBoost Test RMSE:", round(xgb_rmse, 3))

```

```

##
## XGBoost Test RMSE: 2.36

```

```

cat("\nXGBoost Test R²: ", round(xgb_r2, 3), "\n")

```

```

##
## XGBoost Test R²: 0.5

```

8. MODEL COMPARISON — YOUR MODEL vs HPI BASELINE

```
hpi_baseline_df <- test_data %>%
  select(leb_cdc, hpi) %>%
  drop_na()

hpi_baseline_model <- lm(leb_cdc ~ hpi, data = hpi_baseline_df)
hpi_baseline_preds <- predict(hpi_baseline_model, hpi_baseline_df)

hpi_rmse <- sqrt(mean((hpi_baseline_df$leb_cdc - hpi_baseline_preds)^2))
hpi_r2 <- cor(hpi_baseline_df$leb_cdc, hpi_baseline_preds)^2

# Summary comparison table
comparison <- tibble(
  Model = c("HPI Baseline (linear)", "LASSO", "Random Forest", "XGBoost"),
  RMSE = round(c(hpi_rmse, lasso_rmse, rf_rmse, xgb_rmse), 3),
  R_squared = round(c(hpi_r2, lasso_r2, rf_r2, xgb_r2), 3)
) %>%
  arrange(RMSE)

cat("\n=== MODEL COMPARISON ===\n")
```

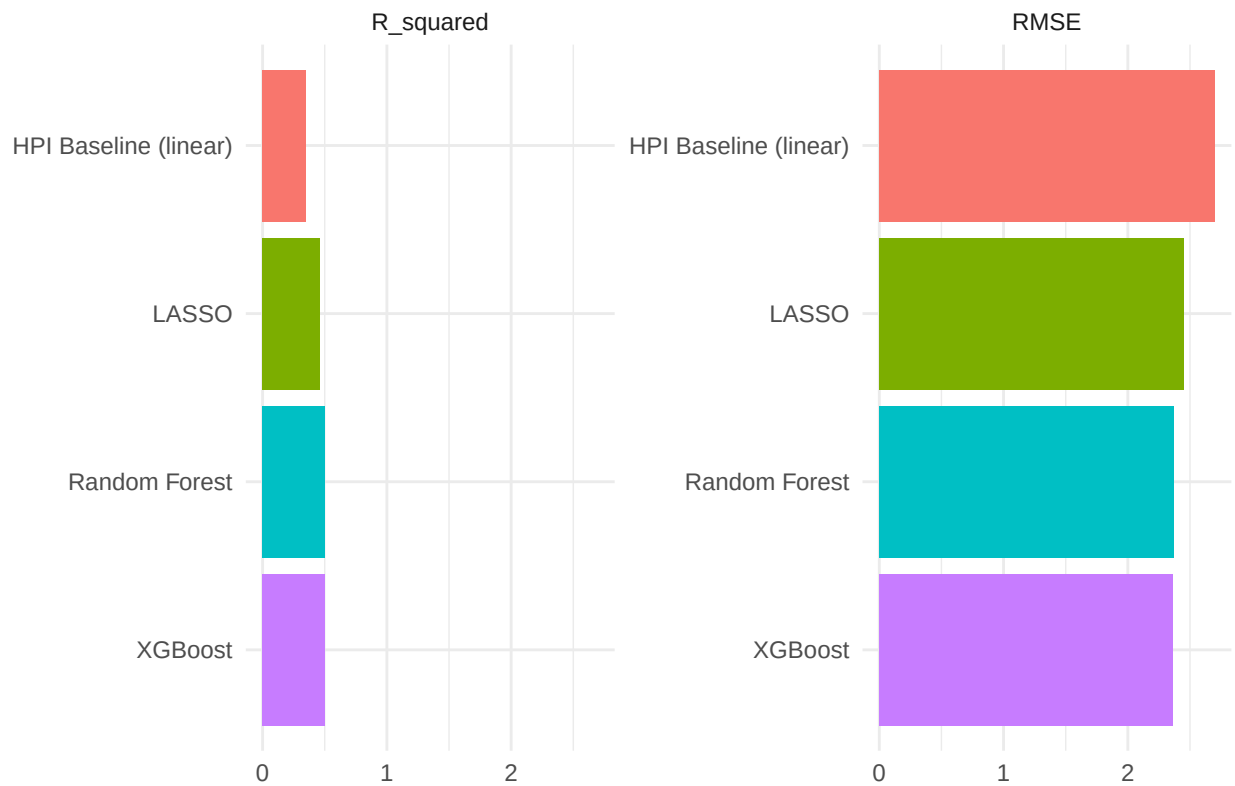
```
##
## === MODEL COMPARISON ===
```

```
print(comparison)
```

```
## # A tibble: 4 x 3
##   Model          RMSE R_squared
##   <chr>         <dbl>   <dbl>
## 1 XGBoost       2.36     0.5
## 2 Random Forest 2.37     0.5
## 3 LASSO         2.45    0.464
## 4 HPI Baseline (linear) 2.70    0.346
```

```
# Visual comparison
comparison %>%
  pivot_longer(cols = c(RMSE, R_squared), names_to = "metric") %>%
  ggplot(aes(x = reorder(Model, value), y = value, fill = Model)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ metric, scales = "free_y") +
  coord_flip() +
  labs(title = "Model Comparison - RMSE and R2",
       x = NULL, y = NULL) +
  theme_minimal()
```


Model Comparison – RMSE and R²

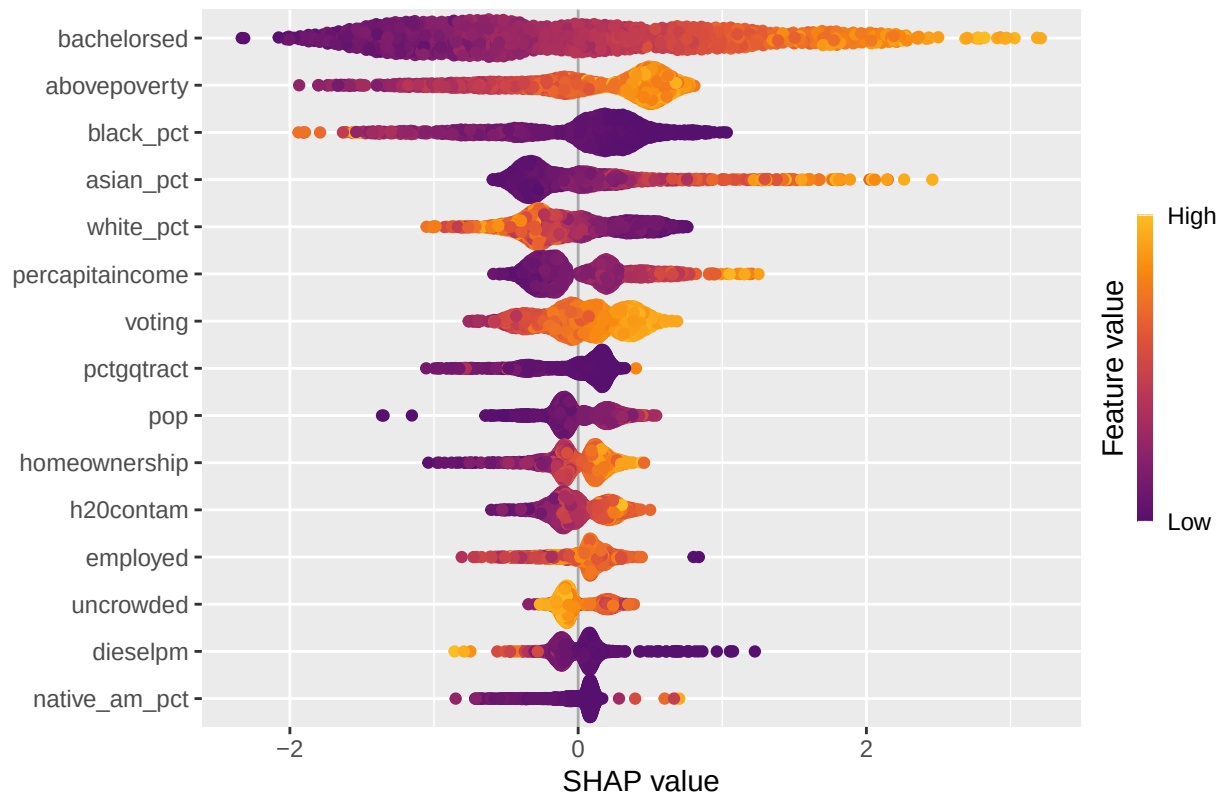


8. SHAP

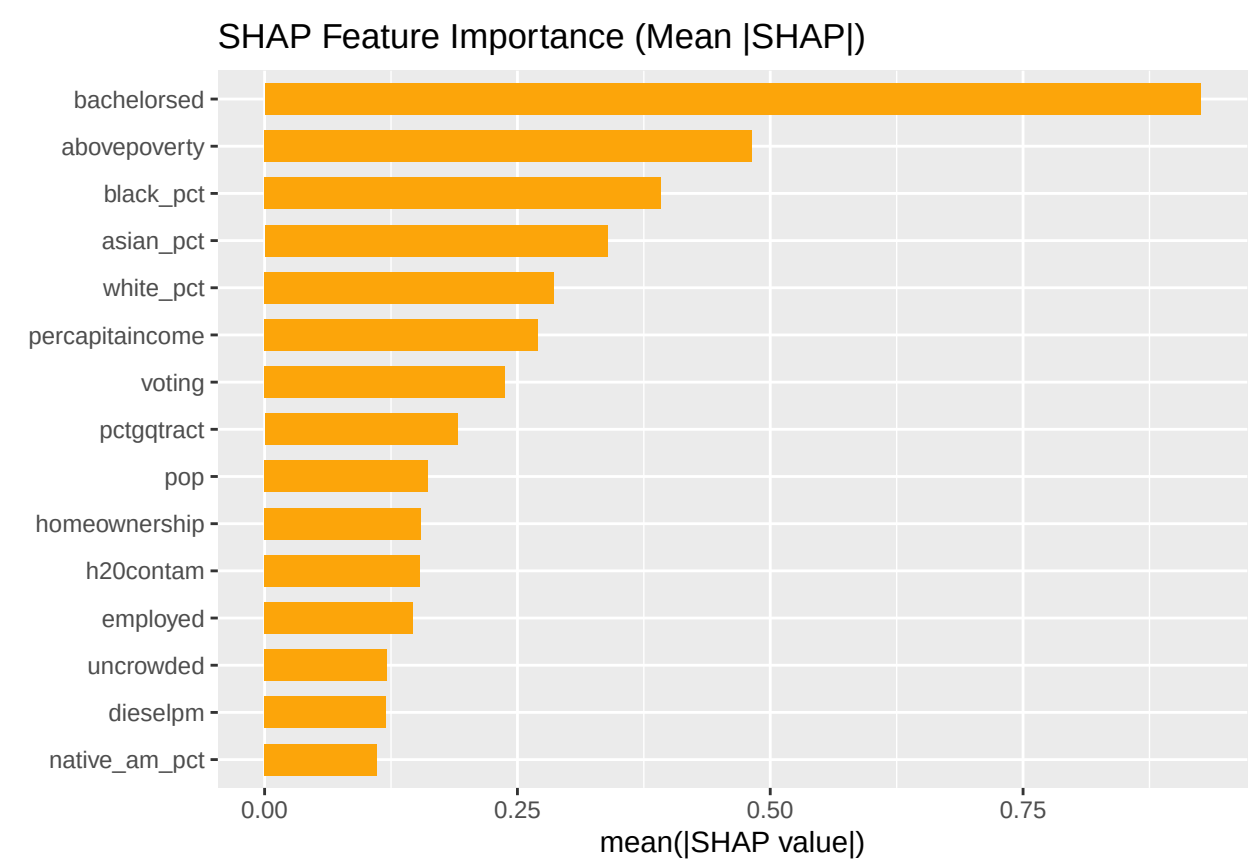
```
shap <- shapviz(xgb_model, X_pred = xgb_test_x)

sv_importance(shap, kind = "beeswarm") +
  labs(title = "SHAP Beeswarm - Feature Impact on Life Expectancy")
```

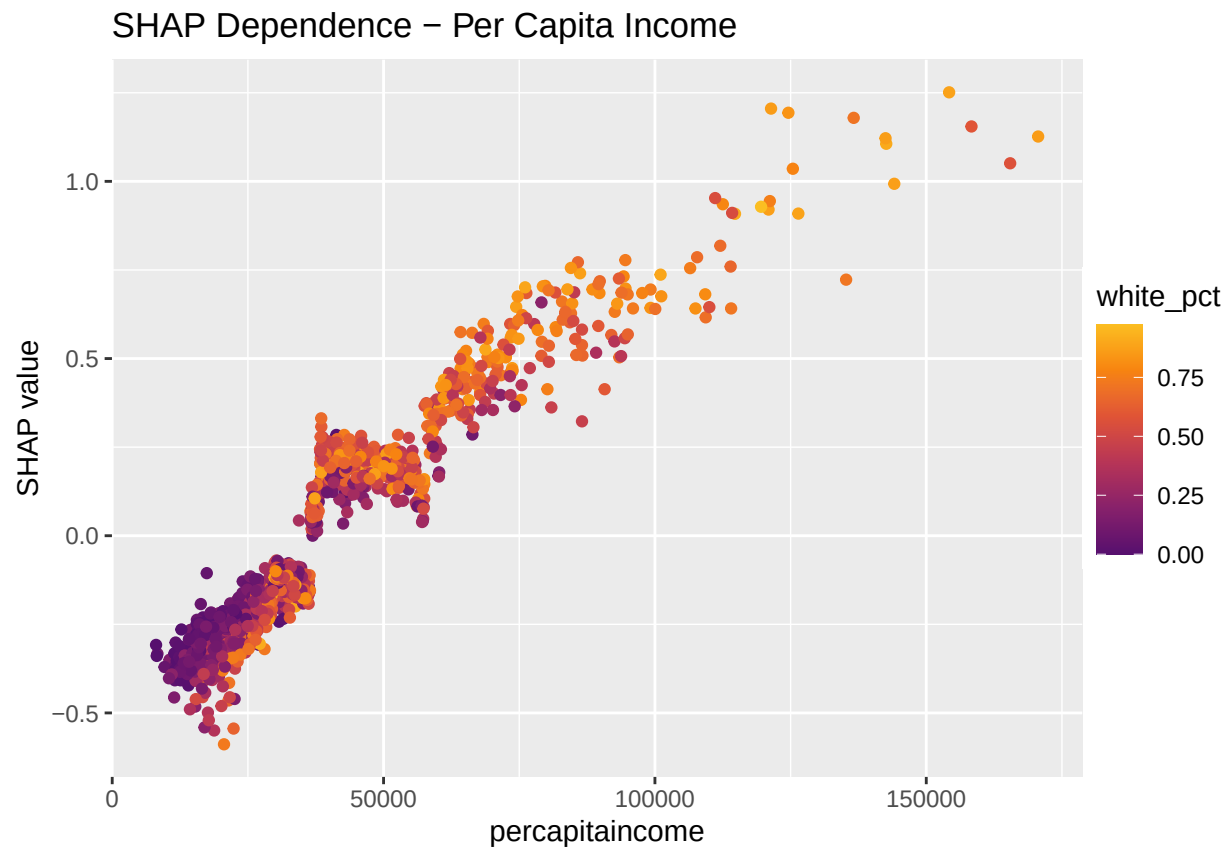
SHAP Beeswarm – Feature Impact on Life Expectancy



```
sv_importance(shap, kind = "bar") +  
  labs(title = "SHAP Feature Importance (Mean |SHAP|)")
```



```
sv_dependence(shap, v = "percapitaincome") +  
  labs(title = "SHAP Dependence - Per Capita Income")
```



```
sv_waterfall(shap, row_id = 1) +  
  labs(title = "SHAP Waterfall - Single Tract Breakdown")
```

SHAP Waterfall – Single Tract Breakdown

